

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №8
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Гибридный алгоритм оптимизации.

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение гибридного алгоритма оптимизации для поиска точек минимума заданных функций.

Гибридный алгоритм оптимизации. Опыт решения сложных задач глобальной оптимизации показывает, что один алгоритм (будь то классический или популяционный) далеко не всегда успешно справляется с задачей.

Гибридные (комбинированные) алгоритмы позволяют объединять сильные стороны разных методов: один алгоритм может компенсировать недостатки другого. В настоящее время разработка гибридных популяционных алгоритмов — одно из основных направлений повышения эффективности глобального поиска. Иногда такие подходы называют гиперэвристиками.

Существует множество способов комбинирования алгоритмов оптимизации, поэтому возникла отдельная проблема их классификации.

Одноуровневая классификация Ванга. Согласно классификации Ванга, гибридные алгоритмы делятся на три основные категории:

1. Вложенные — один алгоритм вложен внутрь другого.
2. Препроцессор/постпроцессор — алгоритмы работают последовательно.
3. Коалгоритмы — алгоритмы работают параллельно, обмениваясь информацией.

Вложенную гибридизацию делят на два типа:

- Высокоуровневая. Алгоритмы слабо связаны между собой. Обычно это комбинация глобального (популяционного) алгоритма и локального алгоритма поиска.
- Низкоуровневая. Алгоритмы сильно интегрированы, так что выделить отдельные компоненты практически невозможно — получается новый алгоритм.

Гибридизация типа «Препроцессор/Постпроцессор». Алгоритмы работают последовательно: первый алгоритм выполняет грубый глобальный поиск, второй — точное локальное уточнение найденного решения.

В рамках этого типа выделяют два класса:

- Последовательная гибридизация — фиксированный порядок алгоритмов (самый частый случай).
- Конвейерная гибридизация — более двух алгоритмов, порядок их включения может динамически определяться в процессе решения задачи.

Коалгоритмическая гибридизация. Алгоритмы работают параллельно (хотя бы на логическом уровне) и обмениваются информацией в процессе поиска.

Классификация Ванга не различает гибридные алгоритмы по тому, выполняются они последовательно или параллельно.

В рамках данной работы был реализован гибридный последовательный алгоритм оптимизации, объединяющий генетический алгоритм и алгоритм градиентного спуска. Гибридный алгоритм заключается в применении для решения задачи сначала генетического алгоритма, а затем алгоритма градиентного спуска, причём в качестве стартовых точек для алгоритма градиентного спуска выбираются лучшие точки, полученные в ходе работы генетического алгоритма.

Таблица сравнения работы алгоритмов, содержащая данные о найденных значениях целевых функций приведена ниже (каждый алгоритм был запущен по 50 раз на каждой функции):

	Sphere			Rastrigin			Rosenbrock		
	Mean	Min	Max	Mean	Min	Max	Mean	Min	Max
Gradient Descent	0.000259	0.000042	0.000567	13.411964	0.000000	40.792967	1.004320	0.001101	3.363085
Genetic Algorithm	0.064507	0.000036	0.586624	1.908061	0.010475	5.395339	0.605372	0.012763	2.242120
Hybrid Algorithm	0.000001	0.000000	0.000012	1.392942	0.000000	7.959662	0.418155	0.003130	1.428036

По таблице видно, что в среднем гибридный алгоритм выдавал самое лучшее (самое минимальное) значение целевой функции по сравнению с генетическим алгоритмом и алгоритмом градиентного спуска, применёнными для решения задачи оптимизации по отдельности.

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки ruqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

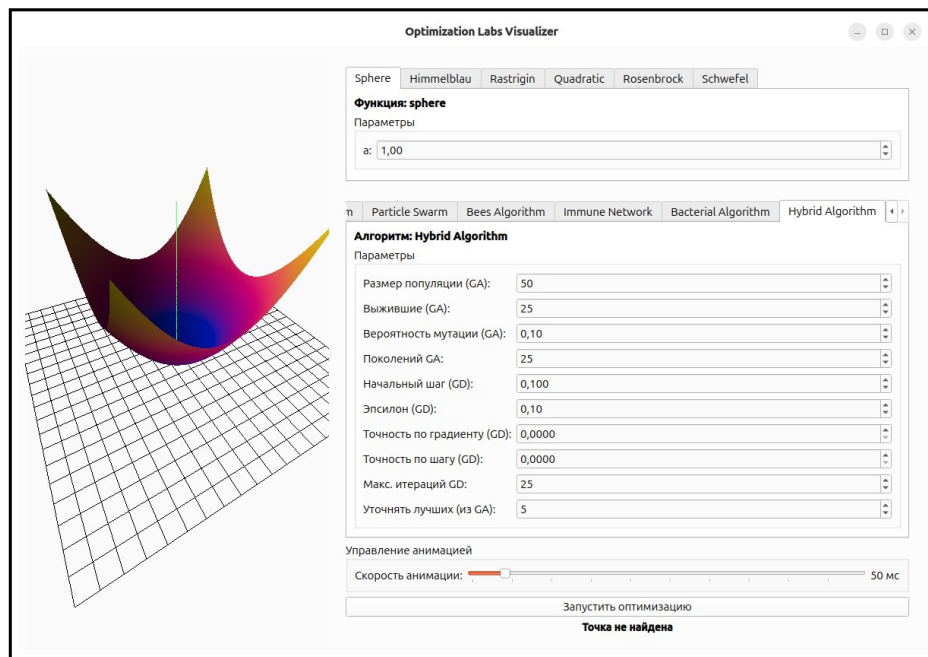


Рисунок 1 — Главное окно приложения

2. В списке функций доступны функции Розенброка, Химмельбау и Растригина (рисунок 2).

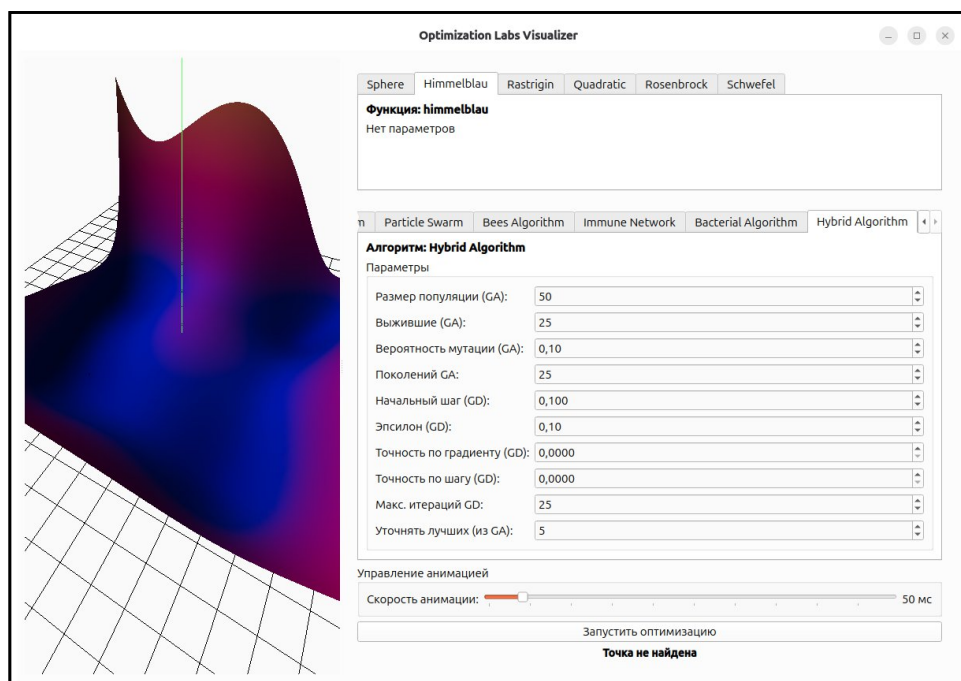


Рисунок 2 — Доступные функции

3. Среди алгоритмов оптимизации доступен гибридный алгоритм (рисунок 3).

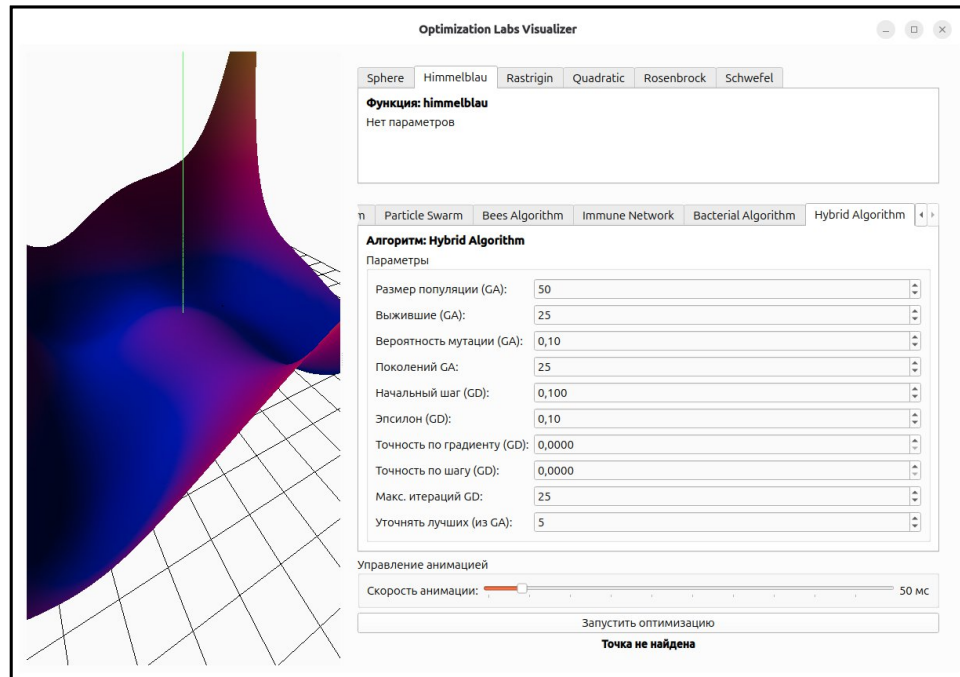


Рисунок 3 — Алгоритм оптимизации

4. Запустим алгоритм поиска минимума функции Розенброка из случайной точки (рисунок 4).

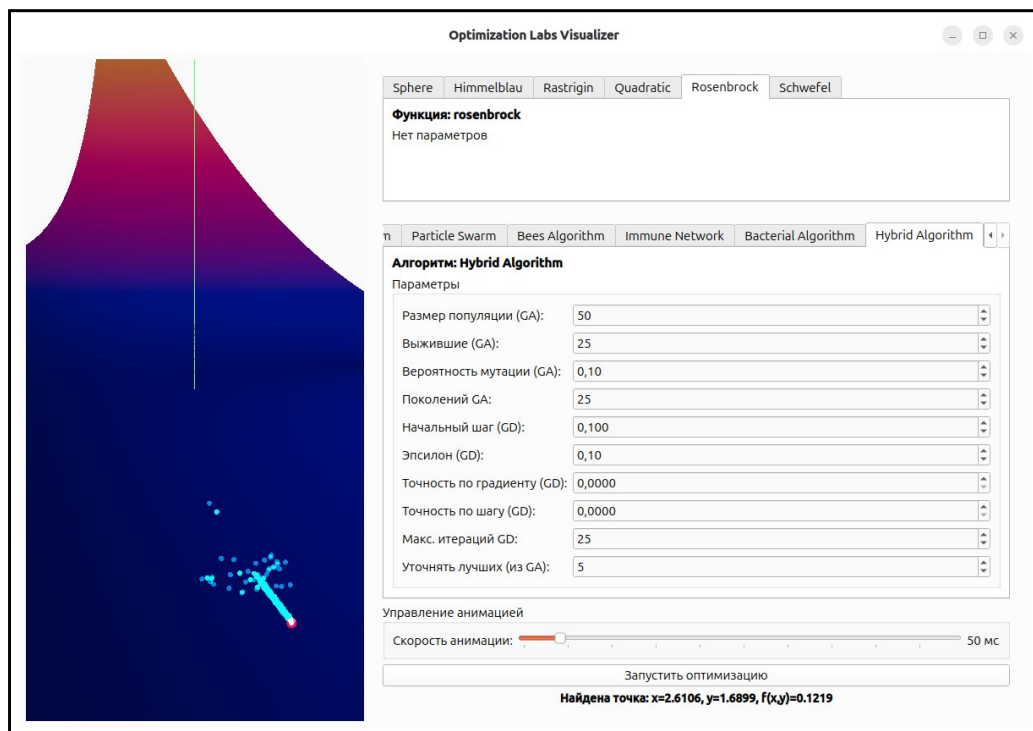


Рисунок 4 — Запуск алгоритма оптимизации

5. Запустим алгоритм поиска минимума функции Растригина из случайной точки (рисунок 5).

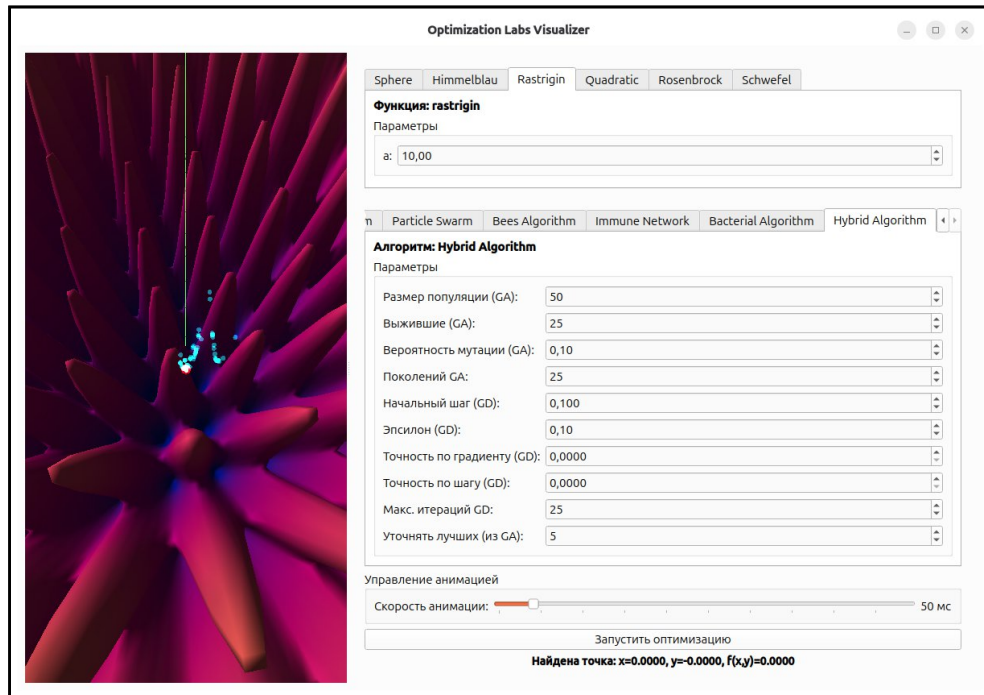


Рисунок 5 — Запуск алгоритма

6. Запустим алгоритм оптимизации с изменёнными параметрами (рисунок 6).

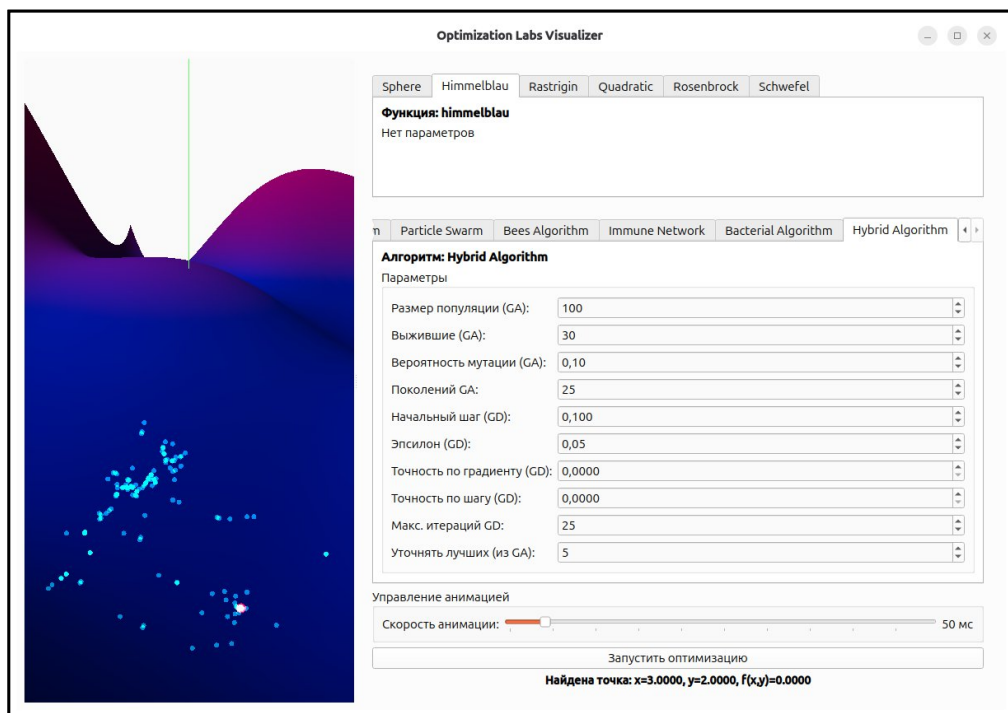


Рисунок 6 — Оптимизация с изменёнными параметрами

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение гибридного алгоритма оптимизации для поиска минимума заданной функции.

Листинг программы.

```
import numpy as np
from .genetic_algorithm import genetic_algorithm
from .gradient_descent import gradient_descent

def hybrid_algorithm(
    func,
    x_range: tuple,
    y_range: tuple,
    # GA params
    pop_size: int = 50,
    die_size: int = 25, # survived_size
    p_mutation: float = 0.1,
    ga_generations: int = 25, # was 100
    # GD params
    t: float = 0.1,
    eps: float = 0.1,
    eps1: float = 1e-4,
    eps2: float = 1e-6,
    M: int = 25, # was 100
    # Hybrid params
    num_refine_best: int = 5, # how many best from GA to refine
    clear_view=None,
    **func_params
):
    """
    Гибридный алгоритм: Генетический (препроцессор) -> Градиентный спуск (постпроцессор)
    Последовательная гибридизация типа препроцессор/постпроцессор
    """
    # Phase 1: Genetic Algorithm (global search)
    ga_gen = genetic_algorithm(
        func, x_range, y_range,
        pop_size, die_size, p_mutation, ga_generations,
        clear_view, **func_params
    )

    final_population = None
    for ga_step in ga_gen:
        # Yield GA steps for visualization
        yield ga_step # list of points from GA
        final_population = ga_step # last one is the final sorted population

    if final_population is None or len(final_population) == 0:
        # Fallback
        yield [ (np.random.uniform(*x_range), np.random.uniform(*y_range)) ]
        return

    # Sort final population by fitness (minimization)
    sorted_pop = sorted(
        final_population,
        key=lambda p: func(*p, **func_params)
    )
```

```

# Take top N best individuals for local refinement
to_refine = sorted_pop[:max(1, min(num_refine_best, len(sorted_pop)))]

# Phase 2: Gradient Descent from each best point
all_refined = []
for start_point in to_refine:
    x0, y0 = start_point
    try:
        gd_gen = gradient_descent(
            func, x0, y0, t, eps, eps1, eps2, M, **func_params
        )
        for gd_step in gd_gen:
            # Yield GD refinement steps
            yield [gd_step[0]] # single point from GD
            all_refined.append(gd_step[0])
    except Exception:
        # If GD fails, keep original
        all_refined.append(start_point)

# Final yield with best refined point
if all_refined:
    best_refined = min(all_refined, key=lambda p: func(*p, **func_params))
    yield [best_refined]
else:
    yield [sorted_pop[0]]

```